

ITF31719-1 26V Bildeanalyse: Final Project

Jon Åsbjørn Hauge

June 5, 2026

1 Introduction

For this project I have chosen to do an object detection and classification model with a YOLO implementation on an animal dataset. I was originally planning to code a full YOLO implementation, but after doing a bit of research on the topic and what is necessary to complete a YOLO implementation, I decided to use Ultralytics YOLO26 implementation for the project given my time constraints and other commitments this semester. My main reasoning for choosing this project is that object detection and how to use a YOLO implementation is a topic I am very interested in learning more about. Secondly I also think this is a project that would be sufficiently challenging for me to further my understanding in the field of image analysis and machine learning.

2 Dataset description

For the project I have used an animal dataset from Kaggle, the original dataset can be found at the link in the reference (Kaggle (antoreepjana), n.d.). Since this is a large dataset with 80 different animal classes and a lot of images, I have chosen to only use a partial part of the dataset to make the training of the model a bit easier to manage. If you look at figure 1 you can see the distribution of the animal classes and the count of images that has been included per animal in the dataset. I have chosen to use a minimum of 100 images per animal class and not more than a maximum of about 220 images per class to keep the dataset relatively balanced.

2.1 Dataset partition

To train and analyse the performance of the YOLO model I have chosen to split the dataset into three subsets with a training, validation and testing partition with a

randomized selection of images for each of the subsets. In the subset I have chosen to use a distribution per animal class of about 55% for the training partition, 15% for the validation partition and 10% for the testing partition. This has been done so that each of the subsets have a different composition of images to better understand the how generalizable the results of the models is. If you look at figures 2, 3 and 4 you can see the distribution and count of the animal classes in the subsets.

3 Setup and training of Ultralytics YOLO26

For this project I decided to use Ultralytics YOLO26 to preform the object detection and classification of the animal datasets. Based on the recommendations in the training guide I have use a pretrained model since this reduces both the required time and dataset size for the training process of the model. Further I used the “yolo26s.pt” pretrained model since this is the recommended model size for my hardware setup (Ultralytics, n.d.-a).

Since the Ultralytics implementation already has well refined default settings and automatic features for selecting optimal settings for options like optimizer and learning rate, I have not to use any significant time tuning and optimizing the settings for the models (Ultralytics, n.d.-d). To make sure it is possible to reproduce the results from the training process, I have used the setting for “deterministic=True” and a specified a specific random seed for each of the different dataset splits (Ultralytics, n.d.-b).

During the training process the Ultralytics YOLO26 implementation has a feature that automatically saves the weights of the best model during the training process, so even if the model starts to deteriorate during the training the best model is saved (Ultralytics, n.d.-f). After the training process I used the weights of the best model for the different dataset splits to run a validation on both the validation and testing splits of the datasets to use in the analysis of the final results.

3.1 Analysis of loss progression during training

If we look at figures 5, 6 and 7 for the loss progression during training of subset 1, 2 and 3 respectively, we can see that the classification loss (cls_loss) already after 30 to 40 epochs starts to plateau both for the training and validation partitions for all the subsets. This indicates that the model dose not seem improve significantly for classification of the objects after this point. This is also supported by the progression in the results for the metrics (figures 5, 6, 7), which also dose not seem to

improve much after that point. We can also see that the progression graphs matches well between the training and validation partitions for all the subsets, which indicates that the model is not over or underfitting for the classification part of the model.

If we look at progression for the the box loss results we can see that the box loss has a steady decline for both the training and validation partitions through most of the epochs, but for the validation partitions the decline seem to taper off towards the last quarter of the epochs. This indicates that the model is starting to overfit in the training for the bounding box detection. We can also see that there is a sudden drop off in the box loss results for the training partitions in the last few epochs, that is not reflected in the validation partitions. This clearly indicates that the model is overfitting at this point.

When we look at the loss progression for both classification and the bounding boxes together we see that the amount of epochs we have selected for the training of the model is sufficient and if anything we could probably get very similar results with a smaller amount of epochs.

I have not included the `dlf.loss` in my analysis since YOLO26 documentation indicates that this has been removed from the loss calculations (Ultralytics, n.d.-e).

4 Results and Analysis

For the YOLO26 model the main performance metrics for class prediction is precision, recall, mAP50 and mAP50-95. Precision is a measure of how accurate the predictions that are made is, recall is a measure of how well a the model detects all images of a given class. The metric mAP50 is a measure of mean average precision given an IoU threshold of 0.50, this metric measures the performance both of the class prediction and the bonding box prediction. The IoU threshold of 0.50 indicates that the metric requires a minimum of 50% overlap with the provided ground truth bounding box label. The mAP50-95 metric is a stricter version of the mAP50 metric and measures mean average precision over IoU thresholds ranging from 0.50 to 0.95 (Ultralytics, n.d.-c).

For the the first subset we get joint results of all the classes with a precision of 0.803 and 0.849 for the validation and test partitions respectively, for recall we get 0.808 and 0.796 respectively, for mAP50 we get 0.837 and 0.84 respectively and for mAP50-95 we get 0.725 and 0.727 respectively (figures 8, 9).

For the the second subset we get joint results of all the classes with a precision of 0.893 and 0.826 for the validation and test partitions respectively, for recall we get 0.777 and 0.76 respectively, for mAP50 we get 0.844 and 0.811 respectively and for mAP50-95 we get 0.718 and 0.712 respectively (figures 10, 11).

For the the third subset we get joint results of all the classes with a precision of 0.788 and 0.824 for the validation and test partitions respectively, for recall we get 0.817 and 0.75 respectively, for mAP50 we get 0.829 and 0.824 respectively and for mAP50-95 we get 0.698 and 0.713 respectively (figures 12, 13).

As we can see from the results the performance between the subsets is relatively similar, especially if we look at the mAP50 and mAP50-95 metrics. These metrics is also having a very similar performance between the validation and test partitions of the subsets, which indicate that performance of the model i translating well to unseen images.

The difference in performance for between mAP50 and mAP50-95 for all the subsets indicate that the model is having more difficulty accurately predicting the classes when we use an higher IoU threshold than 0.50, this can potentially indicate that the model is struggling to achieve a high IoU overlap with the provided ground truth label for the bounding boxes.

If we look at the figures with the complete results (8, 9, 10, 11, 12, 13) we can see that there is some of the individual classes is having significantly worse results than the other classes, this is especially apparent for the cheetah and leopard classes for all of the subsets and for the sheep class for the first and second subsets. This is something we will investigate further in the analysis if the confusion matrix's.

4.1 Confusion Matrix's

For the confusion matrix's i have chosen to focus the analysis on the normalized matrix's since the normalized values are a bit easier to interpret between the classes when we have different amounts of images for the classes. When we look at the matrix's for both the validation and testing partitions we can see that there is a lot more variation her in the results between the partitions and subsets for individual classes then was apparent when we looked at the overall results for all the classes (figures 14, 15, 16, 17, 18, 19).

If we look at the classes we noted earlier for having a weak performance, we can see that the model is mixing the predictions for the cheetah and leopard classes,

it seems that the model is predicting a lot of cheetah where leopard is the correct class. This could indicate that the model does not have a large enough dataset to properly distinguish between these classes since they are relatively similar animals. It would still be prudent to investigate the dataset for wrongly classified images between these classes since there is a large amount of wrong predictions between the classes.

For the sheep class it seems that the biggest issue is that the model is struggling to detect all the sheep objects, the results show that the model predicts background for several of the images where the correct prediction is the sheep class. This could indicate that the model needs more training data for the sheep class and preferably with more images where it is more difficult to separate the background and the sheep.

If we look at the background predictions we can see the the model is struggling to detect the objects in several of the the images. This amplifies our suspicion from previously when we were comparing the mAP50 and mAP50-95 metrics, that the model is under performing when it comes to detection of the bounding box predictions. We can also see that the model is having some issues with detecting objects where there should not be any objects. This likely indicates that the model would benefit from a larger dataset with more images to train on to improve the bounding box detections. The progression of the loss function results could also indicate this, since the progression for the bounding box loss generally improved for the validation loss results through the epochs, while class label loss quite early started to plateau (figures 5, 6, 7).

4.2 F1-Confidence, Precision-Confidence, Recall-Confidence and Precision-Recall Curve's

For this section I have chosen to focus on analyzing the results for the test partitions of the subsets since I feel this is sufficient to show my understanding of how to interpret these curve's.

4.2.1 F1-Confidence Curve's

The F1-confidence curve shows the relationship between the f1-score and IoU confidence threshold for the model. The f1-score is a metric that balances the performance of the precision and recall metrics and therefore is useful when you want to optimize the balanced performance of the model (Ultralytics, n.d.-c). As we can see from the figures (20, 21, 22) all of the subsets have similar average performance for all the classes and the optimal IoU confidence threshold is between

0.3-0.4 for all of the subsets. We can also see that all the subsets would give close to optimal performance for the f1-score in a range between an IoU confidence threshold of 0.1-0.8. This means that depending if you want to optimize the model for precision or recall, you can adjust the setting for the IoU confidence threshold for the model in this range and still achieve relatively similar overall performance.

4.2.2 Precision-Confidence, Recall-Confidence and Precision-Recall Curve's

If we look at figures (23, 24, 25) for the precision-confidence curve's and figures (26, 27, 28) for the recall-confidence curve's we can see how the f1-confidence curve's are combining these into joint curve's. If we first look at the precision-confidence curve's we can see that as the IoU confidence threshold increases the precision for the model predictions also increases, while if we look at the recall-confidence curve's we see the opposite effect where the recall reduces as the IoU confidence threshold increases.

The precision-recall curve's shows the relationship between precision and recall directly and from the figures (29, 30, 31) we can see that if we increase precision performance, recall performance reduces and also the inverse relationship if we increase the recall performance precision performance reduces (Ultralytics, n.d.-c).

4.3 Speed Performance

Typically YOLO implementations is also evaluated on speed performance since one of the main reasons for developing the YOLO implementation was increased speed performance on inferences predictions. Given that I only have been interested in still image performance in this project, I have not included any analysis of speed performance on inferences latency (Ultralytics, n.d.-c).

5 Result summary

The model seems to be performing relatively well given the size of the dataset used in the project, with a performance of 0.825 and 0.717 respectively, for mAP50 and mAP50-95 averaged over all the subsets for the test partitions. The model performance seems to be limited by a couple of weak classes and non optimal bounding box detection in some condition. To improve the performance I would try to expand the dataset with more images for the weak classes to improve the performance on those classes. I would also try to include more images in general to improve the bounding box performance, preferably in more varied conditions so that the model broader base to learn from.

6 Figures

6.1 Dataset description and partition

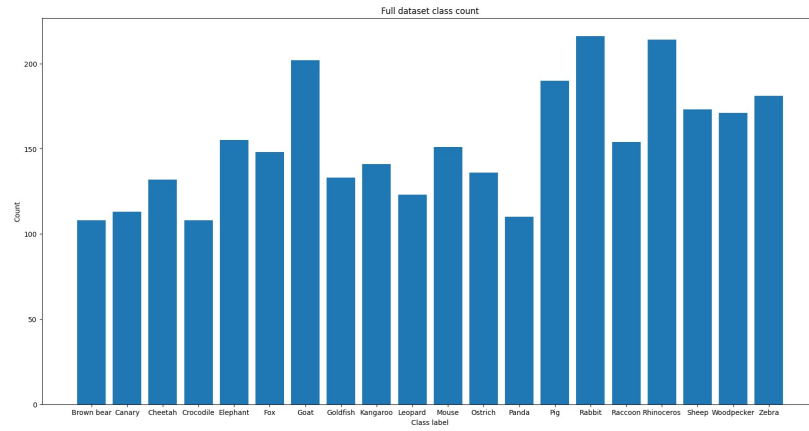


Figure 1: Image of the class count in the full dataset.

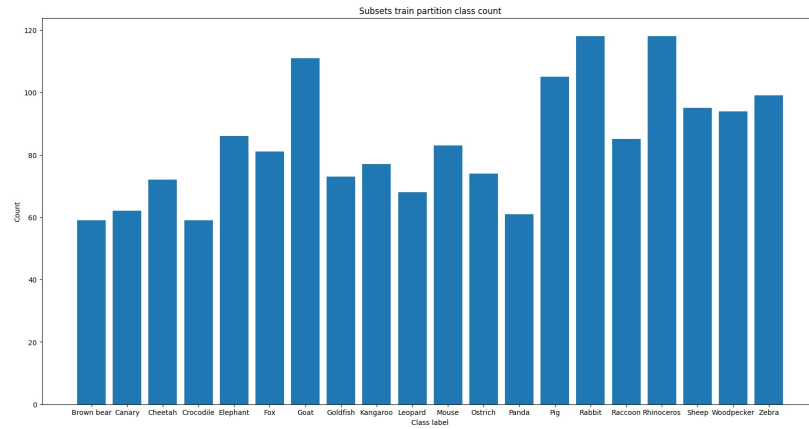


Figure 2: Image of the class count in the train subsets.

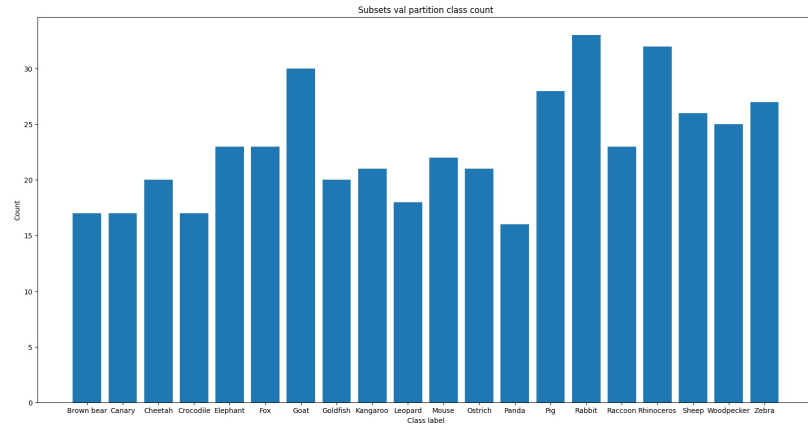


Figure 3: Image of the class count in the validation subsets.

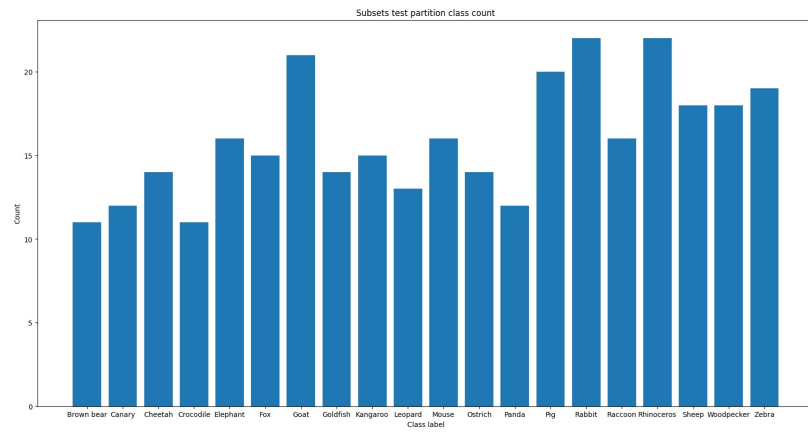


Figure 4: Image of the class count in the test subsets.

6.2 Results progression

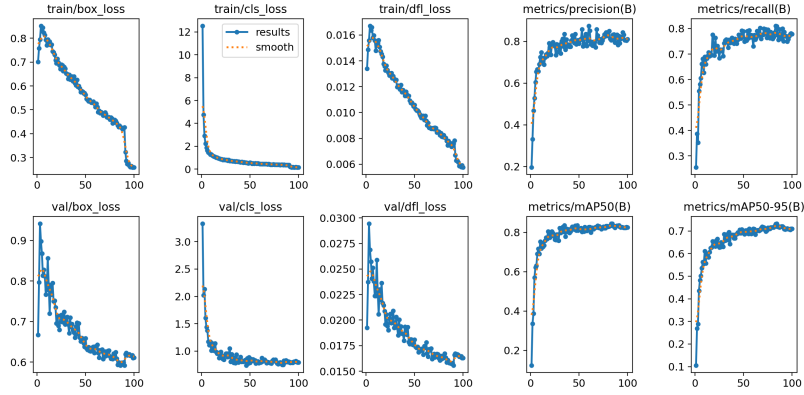


Figure 5: Image of results progression during training for subset 1.

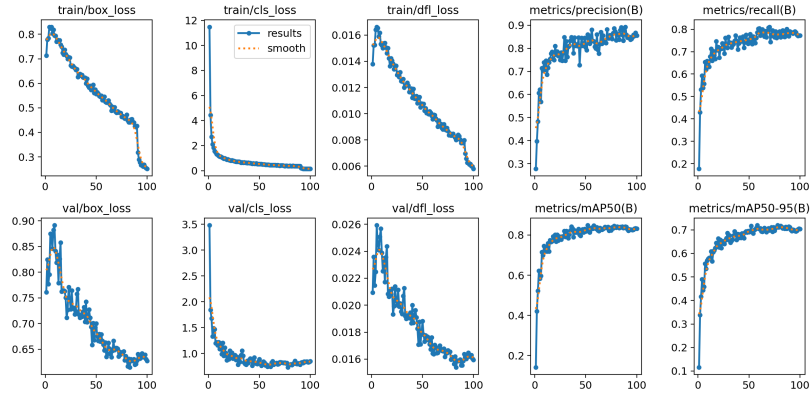


Figure 6: Image of results progression during training for subset 2.

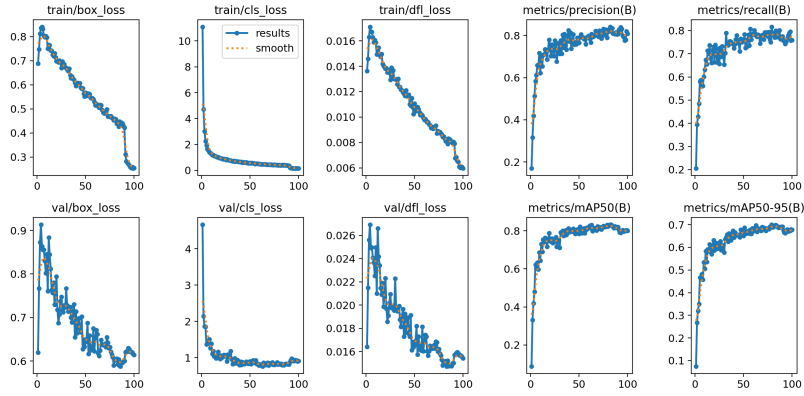


Figure 7: Image of results progression during training for subset 3.

6.3 Results and Analysis

Class	Images	Instances	Box(P)	R	mAP50	mAP50-95)	100%	29/29 13.31t/s 2.290.1s
all	459	515	0.883	0.808	0.837	0.725		
Brown bear	17	18	0.818	0.889	0.917	0.882		
Canary	17	20	0.951	0.9	0.94	0.82		
Cheetah	20	22	0.471	0.609	0.519	0.474		
Crocodile	17	18	0.949	0.778	0.873	0.728		
Elephant	23	23	0.796	0.957	0.982	0.982		
Fox	23	24	0.954	0.865	0.925	0.886		
Goat	30	32	0.712	0.875	0.836	0.69		
Goldfish	20	26	0.841	0.731	0.774	0.584		
Kangaroo	21	24	0.791	0.791	0.848	0.765		
Leopard	18	18	0.501	0.612	0.544	0.466		
Mouse	22	23	0.791	0.66	0.777	0.503		
Ostrich	21	22	0.847	1	0.995	0.875		
Panda	16	16	0.879	0.905	0.924	0.805		
Pig	28	35	0.827	0.743	0.798	0.694		
Rabbit	33	33	0.841	0.788	0.864	0.765		
Raccoon	23	24	0.727	0.778	0.815	0.72		
Rhinoceros	32	35	0.938	1	0.995	0.972		
Sheep	26	48	0.652	0.55	0.62	0.487		
Woodpecker	25	26	0.961	0.951	0.963	0.754		
Zebra	27	36	0.804	0.778	0.838	0.722		

Speed: 0.5ms preprocess, 1.0ms inference, 0.0ms loss, 0.1ms postprocess per image

Figure 8: Screen shoot of validation results for validation partition ("dataset_1_val.ipynb") in subset 1.

Class	Images	Instances	Box(P	R	mAP50	mAP50-95)	100%	20/20	13.4it/s	1.5s0.1s
all	319	356	0.849	0.796	0.84	0.727				
Brown bear	11	11	0.89	0.733	0.981	0.86				
Canary	12	12	0.803	0.917	0.937	0.794				
Cheetah	14	14	0.617	0.857	0.655	0.63				
Crocodile	11	12	0.906	0.806	0.954	0.682				
Elephant	16	17	1	0.88	0.929	0.817				
Fox	15	16	0.929	0.815	0.897	0.859				
Goat	21	24	0.952	0.826	0.855	0.779				
Goldfish	14	18	0.733	0.611	0.675	0.509				
Kangaroo	15	17	1	0.765	0.823	0.683				
Leopard	13	13	0.529	0.462	0.476	0.451				
Mouse	16	17	0.73	0.636	0.672	0.432				
Ostrich	14	16	1	0.802	0.935	0.814				
Panda	12	13	0.953	0.769	0.871	0.797				
Pig	20	26	0.812	0.833	0.844	0.789				
Rabbit	22	25	0.881	0.8	0.941	0.832				
Raccoon	16	18	0.868	0.944	0.936	0.867				
Rhinoceros	22	23	0.865	1	0.985	0.929				
Sheep	18	20	0.649	0.75	0.697	0.583				
Woodpecker	18	19	0.956	0.842	0.911	0.69				
Zebra	19	25	0.911	0.88	0.915	0.754				

Speed: 0.6ms preprocess, 1.8ms inference, 0.0ms loss, 0.2ms postprocess per image

Figure 9: Screen shoot of validation results for test partition ("dataset_1_test.ipynb") in subset 1.

Class	Images	Instances	Box(P	R	mAP50	mAP50-95)	100%	29/29	13.4it/s	2.2s0.1s
all	459	527	0.893	0.777	0.844	0.718				
Brown bear	17	20	1	0.749	0.856	0.782				
Canary	17	19	0.886	0.895	0.901	0.838				
Cheetah	20	20	0.702	0.589	0.543	0.494				
Crocodile	17	18	0.977	0.889	0.942	0.703				
Elephant	23	25	0.884	0.84	0.953	0.82				
Fox	23	24	0.948	0.917	0.979	0.906				
Goat	30	38	0.869	0.816	0.883	0.777				
Goldfish	20	24	0.87	0.75	0.793	0.558				
Kangaroo	21	21	1	0.777	0.94	0.85				
Leopard	18	18	0.643	0.722	0.61	0.526				
Mouse	22	22	0.916	0.497	0.744	0.464				
Ostrich	21	24	0.936	0.833	0.963	0.813				
Panda	16	16	0.927	0.688	0.797	0.726				
Pig	28	34	0.825	0.735	0.828	0.678				
Rabbit	33	34	0.736	0.765	0.795	0.712				
Raccoon	23	24	0.953	0.841	0.883	0.747				
Rhinoceros	32	43	0.969	0.907	0.977	0.888				
Sheep	26	40	0.817	0.559	0.591	0.44				
Woodpecker	25	28	1	0.908	0.964	0.755				
Zebra	27	35	1	0.863	0.948	0.885				

Speed: 0.6ms preprocess, 2.1ms inference, 0.0ms loss, 0.1ms postprocess per image

Figure 10: Screen shoot of validation results for validation partition ("dataset_2_val.ipynb") in subset 2.

Class	Images	Instances	Box(P)	R	mAP50	mAP50-95)	100%	20/20	13.6it/s	1.5s0.1s
all	319	365	0.826	0.76	0.811	0.712				
Brown bear	11	11	0.756	1	0.965	0.947				
Canary	12	15	1	0.849	0.93	0.855				
Cheetah	14	15	0.447	0.6	0.394	0.394				
Crocodile	11	11	0.958	1	0.995	0.858				
Elephant	16	18	0.998	0.833	0.835	0.733				
Fox	15	15	0.873	0.933	0.985	0.894				
Goat	21	23	0.622	0.717	0.612	0.512				
Goldfish	14	18	0.754	0.778	0.75	0.496				
Kangaroo	15	20	0.929	0.65	0.763	0.662				
Leopard	13	13	0.384	0.241	0.418	0.378				
Mouse	16	16	0.766	0.5	0.717	0.483				
Ostrich	14	17	1	0.939	0.945	0.826				
Panda	12	12	0.956	0.833	0.898	0.869				
Pig	20	32	0.798	0.494	0.739	0.671				
Rabbit	22	25	0.738	0.675	0.779	0.683				
Raccoon	16	17	0.901	0.824	0.935	0.913				
Rhinoceros	22	23	1	0.949	0.99	0.969				
Sheep	18	24	0.748	0.62	0.632	0.449				
Woodpecker	18	18	1	0.857	0.989	0.8				
Zebra	19	22	0.899	0.989	0.942	0.849				

Speed: 0.5ms preprocess, 1.8ms inference, 0.0ms loss, 0.1ms postprocess per image

Figure 11: Screen shoot of validation results for test partition ("dataset_2_test.ipynb") in subset 2.

Class	Images	Instances	Box(P)	R	mAP50	mAP50-95)	100%	29/29	13.8it/s	2.1s0.1s
all	459	514	0.788	0.817	0.829	0.698				
Brown bear	17	18	0.772	0.778	0.738	0.678				
Canary	17	18	0.764	1	0.995	0.745				
Cheetah	20	20	0.45	0.6	0.386	0.349				
Crocodile	17	17	0.888	0.932	0.983	0.852				
Elephant	23	24	0.874	1	0.978	0.842				
Fox	23	24	0.803	0.792	0.884	0.719				
Goat	30	35	0.674	0.886	0.848	0.728				
Goldfish	20	23	0.882	0.739	0.844	0.595				
Kangaroo	21	23	0.723	0.87	0.875	0.75				
Leopard	18	18	0.586	0.55	0.48	0.458				
Mouse	22	23	0.668	0.525	0.546	0.362				
Ostrich	21	21	0.876	1	0.952	0.785				
Panda	16	16	0.87	0.812	0.916	0.786				
Pig	28	32	0.708	0.757	0.835	0.788				
Rabbit	33	36	0.788	0.778	0.851	0.782				
Raccoon	23	25	0.838	0.84	0.85	0.695				
Rhinoceros	32	36	0.86	0.856	0.925	0.907				
Sheep	26	40	0.852	0.718	0.834	0.539				
Woodpecker	25	25	0.952	0.96	0.97	0.747				
Zebra	27	40	0.93	0.95	0.968	0.858				

Speed: 0.5ms preprocess, 1.8ms inference, 0.0ms loss, 0.1ms postprocess per image

Figure 12: Screen shoot of validation results for validation partition ("dataset_3_val.ipynb") in subset 3.

Class	Images	Instances	Box(P)	R	mAP50	mAP50-95): 100%	20/20 13.9it/s 1.450.1s
all	319	358	0.843	0.75	0.824	0.713	
Brown bear	11	12	0.901	0.833	0.906	0.887	
Canary	12	13	0.916	0.923	0.925	0.873	
Cheetah	14	15	0.591	0.733	0.589	0.576	
Crocodile	11	11	1	0.721	0.936	0.792	
Elephant	16	21	0.933	0.661	0.793	0.698	
Fox	15	15	0.886	0.8	0.896	0.887	
Goat	21	22	0.747	0.539	0.772	0.672	
Goldfish	14	18	0.825	0.785	0.893	0.68	
Kangaroo	15	17	0.791	0.647	0.777	0.609	
Leopard	13	13	0.564	0.538	0.445	0.486	
Mouse	16	16	0.725	0.66	0.698	0.448	
Ostrich	14	17	0.921	0.941	0.982	0.813	
Panda	12	12	0.942	0.833	0.898	0.81	
Pig	20	22	0.862	0.727	0.842	0.77	
Rabbit	22	23	0.818	0.587	0.729	0.594	
Raccoon	16	17	0.791	0.834	0.91	0.885	
Rhinoceros	22	25	0.905	0.762	0.9	0.789	
Sheep	18	27	0.797	0.729	0.754	0.642	
Woodpecker	18	19	0.942	0.947	0.942	0.74	
Zebra	19	23	1	0.813	0.901	0.841	

Speed: 0.5ms preprocess, 1.8ms inference, 0.8ms loss, 0.1ms postprocess per image

Figure 13: Screen shoot of validation results for test partition ("dataset_3_test.ipynb") in subset 3.

6.3.1 Confusion matrix's

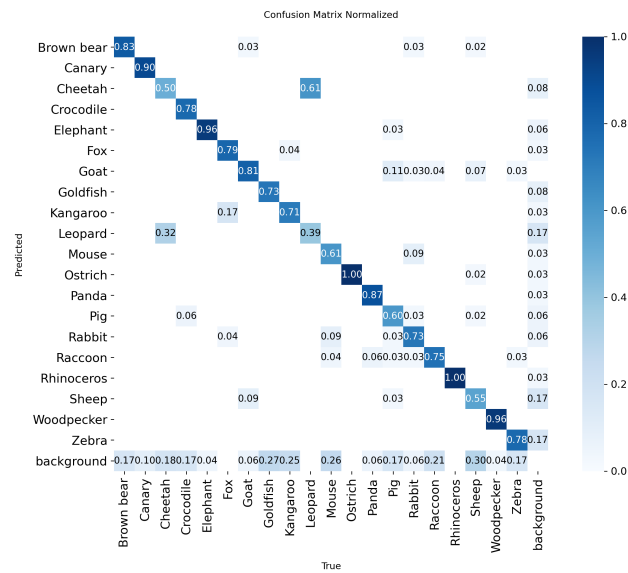


Figure 14: Image of normalized confusion matrix for validation results for validation partition in subset 1.

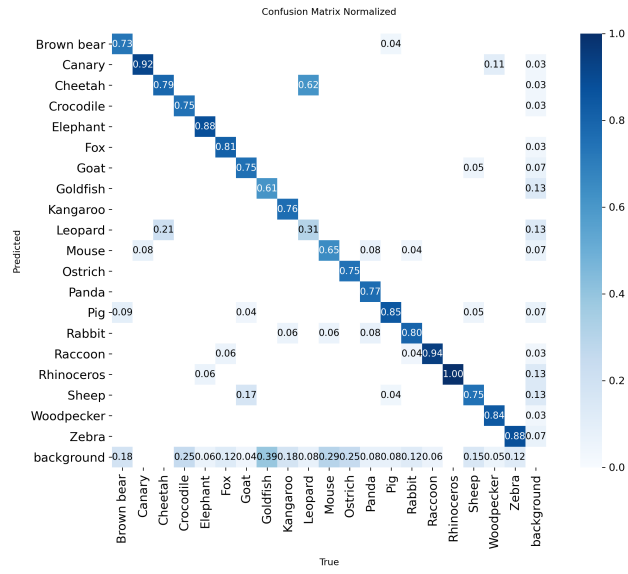


Figure 15: Image of normalized confusion matrix for validation results for test partition in subset 1.

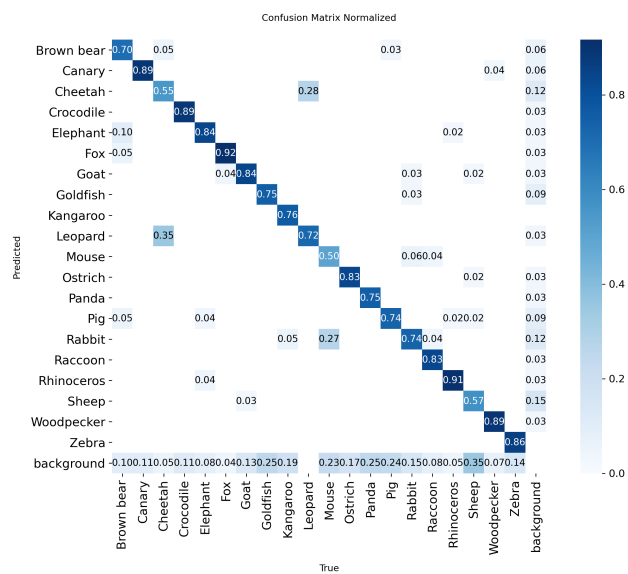


Figure 16: Image of normalized confusion matrix for validation results for validation partition in subset 2.

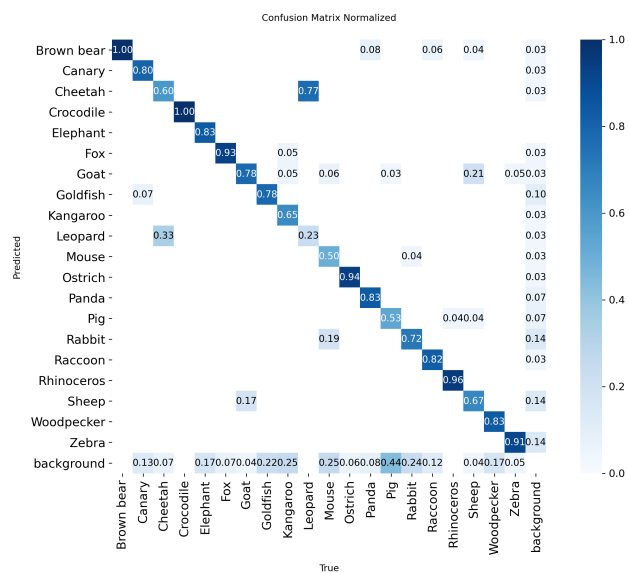


Figure 17: Image of normalized confusion matrix for validation results for test partition in subset 2.

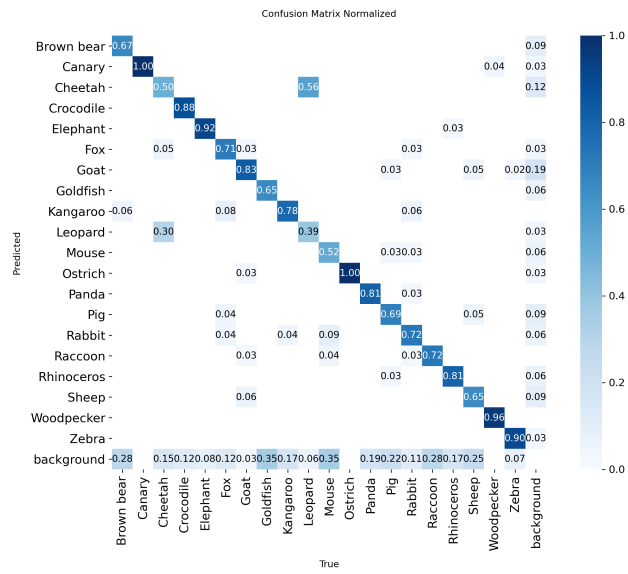


Figure 18: Image of normalized confusion matrix for validation results for validation partition in subset 3.

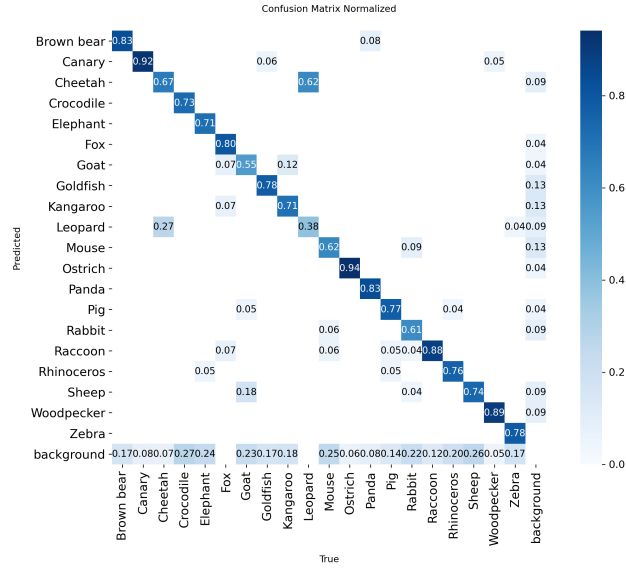


Figure 19: Image of normalized confusion matrix for validation results for test partition in subset 3.

6.3.2 F1-Confidence, Precision-Confidence, Recall-Confidence and Precision-Recall Curve's

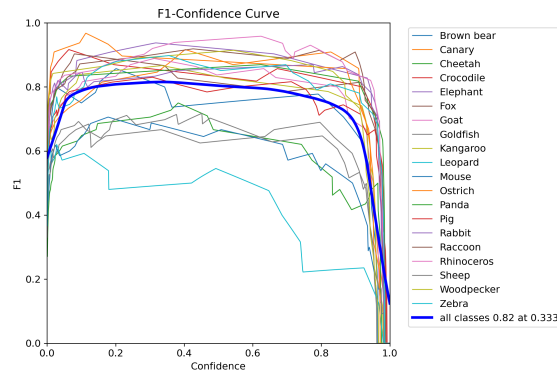


Figure 20: Image of F1-Confidence curve for validation of test partition in subset 1.

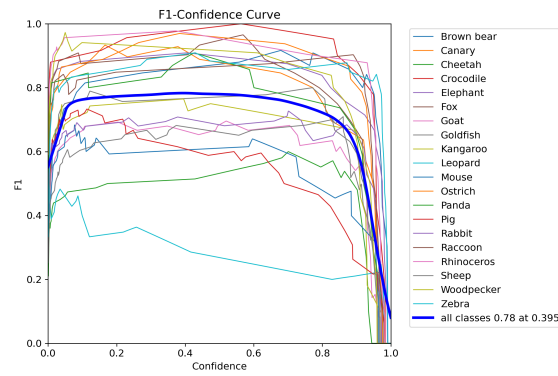


Figure 21: Image of F1-Confidence curve for validation of test partition in subset 2.

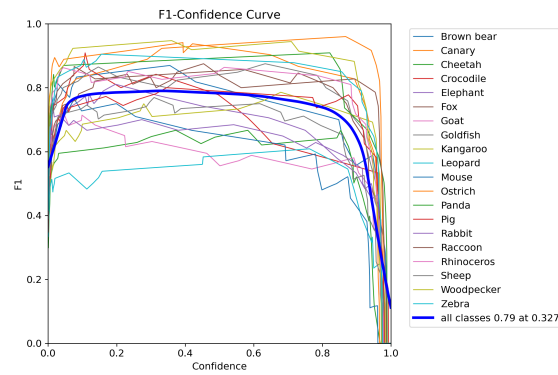


Figure 22: Image of F1-Confidence curve for validation of test partition in subset 3.

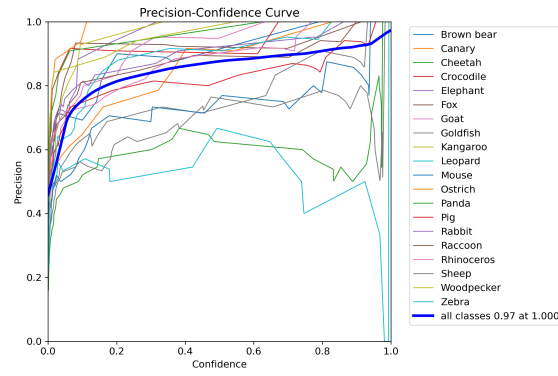


Figure 23: Image of Precision-Confidence curve for validation of test partition in subset 1.

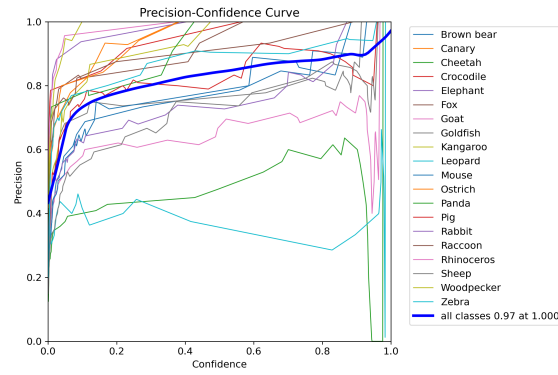


Figure 24: Image of Precision-Confidence curve for validation of test partition in subset 2.

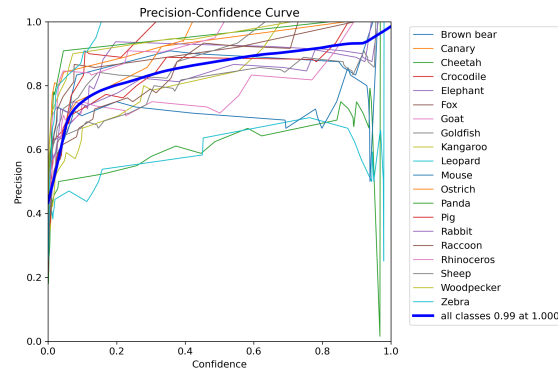


Figure 25: Image of Precision-Confidence curve for validation of test partition in subset 3.

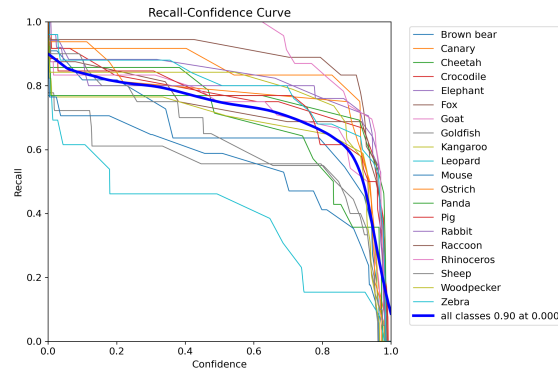


Figure 26: Image of Recall-Confidence curve for validation of test partition in subset 1.

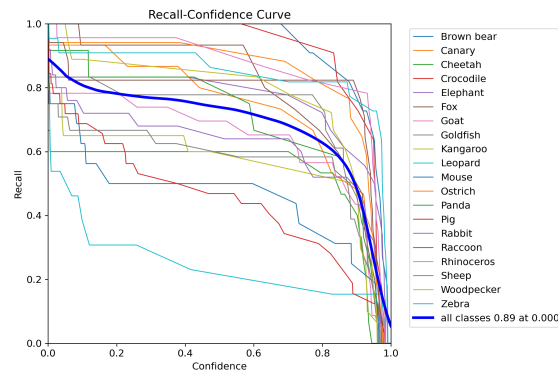


Figure 27: Image of Recall-Confidence curve for validation of test partition in subset 2.

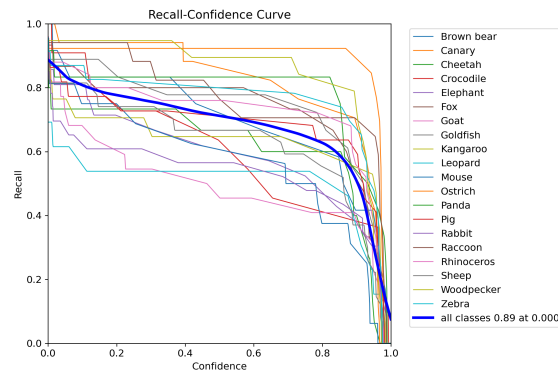


Figure 28: Image of Recall-Confidence curve for validation of test partition in subset 3.

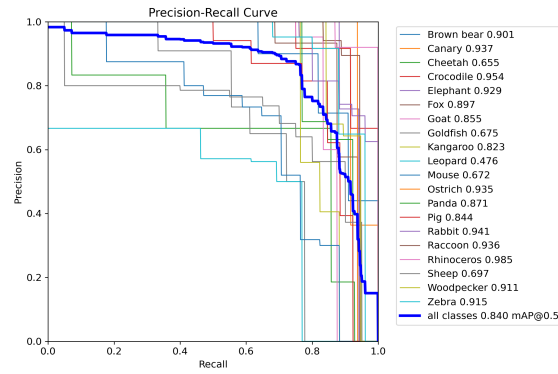


Figure 29: Image of Precision-Recall curve for validation of test partition in subset 1.

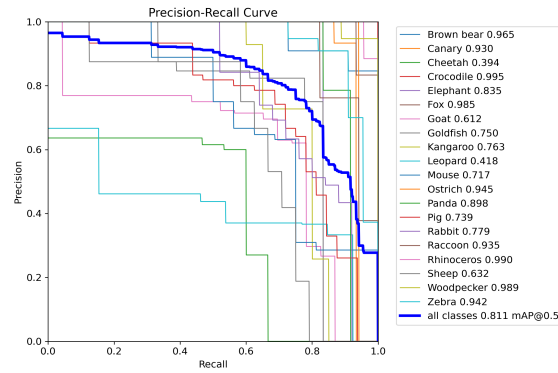


Figure 30: Image of Precision-Recall curve for validation of test partition in subset 2.

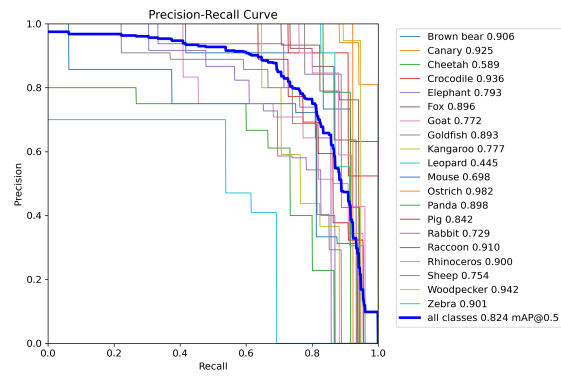


Figure 31: Image of Precision-Recall curve for validation of test partition in subset 3.

References

- Kaggle (antoreepjana). (n.d.). Animals detection images dataset. Retrieved May 25, 2026, from <https://www.kaggle.com/datasets/antoreepjana/animals-detection-images-dataset?resource=download&select=train>
- Ultralytics. (n.d.-a). Choose an ultralytics yolo model size. Retrieved June 3, 2026, from <https://academy.ultralytics.com/courses/train-your-first-yolo/choose-a-model-size>
- Ultralytics. (n.d.-b). Model training with ultralytics yolo. Retrieved June 3, 2026, from <https://docs.ultralytics.com/modes/train#introduction>
- Ultralytics. (n.d.-c). Performance metrics deep dive. Retrieved June 4, 2026, from <https://docs.ultralytics.com/guides/yolo-performance-metrics#interpreting-the-output>
- Ultralytics. (n.d.-d). Train your first model. Retrieved June 3, 2026, from <https://academy.ultralytics.com/courses/train-your-first-yolo/train-your-first-model#outcome>
- Ultralytics. (n.d.-e). Ultralytics yolo26. Retrieved June 4, 2026, from <https://docs.ultralytics.com/models/yolo26#overview>
- Ultralytics. (n.d.-f). Validate and diagnose. Retrieved June 3, 2026, from <https://academy.ultralytics.com/courses/train-your-first-yolo/validate-and-diagnose#outcome>